FACULDADE DE TECNOLOGIA SENAC PORTO ALEGRE – UNISENAC
Escola de Tecnologia da Informação (Escola de TI)
Componente Curricular: Programação Orientada a Objetos
Docente: Prof. Me. Fábio Giulian Marques

🏠 Sistema de Gerenciamento Imobiliário
Imóvel, Proprietário e Corretor 🏢

Objetivos Pedagógicos:

- Fortalecer a compreensão sobre associação entre classes em Python (relacionamento entre objetos).
- Aplicar os conceitos de Programação Orientada a Objetos (POO) em um cenário do mercado real imobiliário.
- Desenvolver a lógica de programação, validações de atributos e métodos de simulação de negócios.

2. ENUNCIADO DO PROBLEMA

Uma imobiliária em expansão precisa de um sistema simples e estruturado para gerenciar a sua carteira de imóveis, seus respectivos proprietários e os corretores responsáveis pelas vendas e locações.

Atualmente, o controle é feito de forma descentralizada, o que gera inconsistências no status dos imóveis e na comissão dos corretores. Seu desafio é implementar um protótipo orientado a objetos em Python que modele os relacionamentos do negócio, permitindo associar proprietários e corretores aos imóveis, alterar o status de disponibilidade e calcular o valor da comissão de venda ou aluguel.

3. REQUISITOS DA APLICAÇÃO

Classes e Atributos Obrigatórios

Classe Proprietario

- **Atributos:** nome (str), cpf (str), telefone (str), email (str).
- **Métodos:**
  - __init__(self, nome, cpf, telefone, email): Construtor para inicialização das informações.
  - exibir_dados(self): Retorna ou imprime os dados formatados do proprietário.

Classe Corretor

- **Atributos:** nome (str), creci (str), percentual_comissao (float — ex: 6.0 para 6%).
- **Métodos:**
  - __init__(self, nome, creci, percentual_comissao): Construtor base.
  - vender_imovel(self, imovel, valor_final): Associa a venda do imóvel, altera o status do imóvel para "Vendido" e calcula o valor da comissão gerada.

”Conteúdo gerado com suporte de Inteligência Artificial para estruturação pedagógica e adaptado, revisado e validado tecnicamente pelo Professor.”

Classe Imovel

- **Atributos:** codigo (int), endereco (str), tipo (str — ex: "Apartamento", "Casa"), valor (float), status (str — "Disponível", "Alugado", "Vendido"), proprietario (Proprietario), corretor (Corretor).
- **Métodos:**
  - __init__(self, codigo, endereco, tipo, valor, proprietario, corretor=None): Define os dados iniciais. O corretor pode ser opcional no cadastro inicial.
  - alterar_status(self, novo_status): Atualiza o status do imóvel validando para que receba apenas valores permitidos.
  - exibir_ficha(self): Imprime o relatório detalhado do imóvel com o nome do proprietário e do corretor associado.

Associação entre as Classes

- Um **Imóvel** pertence a apenas um **Proprietário**.
- Um **Imóvel** possui um **Corretor** responsável designado.
- Um **Proprietário** ou **Corretor** pode estar associado a múltiplos **Imóveis**.

Exercícios de Prática e Execução

1. Crie pelo menos **2 objetos da classe Proprietario** e **2 objetos da classe Corretor**.
2. Crie pelo menos **3 objetos da classe Imovel**, vinculando os proprietários e corretores correspondentes.
3. Simule a transação de venda de um dos imóveis utilizando o método vender_imovel do corretor, exibindo a comissão calculada na tela.
4. Tente alterar o status de um imóvel e imprima a ficha atualizada no console.

”Conteúdo gerado com suporte de Inteligência Artificial para estruturação pedagógica e adaptado, revisado e validado tecnicamente pelo Professor.”

4. DICAS DE IMPLEMENTAÇÃO DO PROFESSOR

Dica 1: Passagem de Objetos como Atributos (Associação)

Lembre-se de que o atributo proprietario da classe Imovel deve receber uma instância (objeto) da classe Proprietario, e não apenas um texto com o nome do cliente.

Dica 2: Calculando a Comissão do Corretor

Dentro da classe Corretor, o método de venda calcula a taxa sobre o valor final do imóvel:

5. POLÍTICA DE AUTORIA, USO DE INTELIGÊNCIA ARTIFICIAL E PLÁGIO

- **Uso Ético de Ferramentas de IA:** O uso de assistentes de IA (como ChatGPT, Claude ou Gemini) é permitido exclusivamente como suporte de estudos, esclarecimento de dúvidas conceituais e busca de exemplos genéricos.
- **Proibição de Cópia e Automação Direta:** É estritamente proibido solicitar que ferramentas de IA resolvam e gerem o código completo do exercício para entrega sem a devida compreensão do estudante.
- **Integridade Acadêmica e Plágio:** O compartilhamento de códigos entre estudantes ou a cópia integral/parcial de trabalhos de colegas caracterizam plágio. Casos identificados resultarão em **Conceito D**, além da abertura de processo disciplinar conforme o Regimento Geral da Instituição.

6. CRITÉRIOS DE AVALIAÇÃO

Categoria	Descrição / Requisitos Atendidos	Conceito Atribuído
Excelente (A)	Aplica integralmente a modelagem de POO (classes, construtores e métodos). Realiza corretamente a associação entre objetos (Imovel, Proprietario e Corretor), implementa todas as validações e testes de execução de forma clara, organizada e seguindo as boas práticas (PEP 8).	**Conceito A**
Bom (B)	Implementa a estrutura principal das classes e a associação de objetos com sucesso. O código executa e atende aos requisitos funcionais da aplicação, apresentando pequenas falhas não estruturais de formatação, validação ou organização.	**Conceito B**
Suficiente (C)	Entregou o código funcional, mas com fragilidades na modelagem POO (ex: atributos definidos de forma rígida/sem parametrização, falta de associação direta entre objetos ou ausência de simulação das transações). Necessita de ajustes para alcançar autonomia na aplicação dos conceitos.	**Conceito C**
Insuficiente (D)	Não atendeu aos requisitos mínimos da atividade, apresentou código com erros de execução não corrigidos, não entregou o projeto no prazo estipulado ou incorreu em plágio/cópia não autorizada.	**Conceito D**

7. ORIENTAÇÕES DE ENTREGA

- **Formato do Arquivo:** Enviar a solução comprimida em arquivo no formato .ZIP.
- **Padrão Obrigatório de Nome do Arquivo:** Imobiliaria_NomeAluno1_NomeAluno2.zip
- **Arquivos Obrigatórios no Pacote:**
  - main.py (contendo as classes ou as chamadas dos métodos de teste).
  - Opcionalmente, arquivos de módulos separados (ex: proprietario.py, imovel.py, corretor.py).
  - *Atenção:* Não inclua pastas de ambientes virtuais (ex: .venv, __pycache__).
- **Local de Envio:** Upload direto na plataforma BçackBoard (Ambiente Virtual de Aprendizagem) até a data estipulada pelo cronograma.